

RTOS using ESP32

Getting started:

In an RTOS workspace, application flows are broken down into independent execution loops called **Tasks**. Unlike sequential bare-metal scripts, tasks are scheduled dynamically by the kernel based on priority levels and timing requests.

In a standard bare-metal C program, execution flows in a single straight line through `main()`. In an RTOS environment, `app_main()` is simply a launching pad. Its only job is to set up the system and spawn independent execution loops called **Tasks**.

Every FreeRTOS task requires two things:

1. **A Task Function:** An infinite loop (`while(1)`) containing the actual work the task needs to execute.
2. **A Call to `xTaskCreate()`:** The registration function that tells the FreeRTOS scheduler to manage this task.

Why is `vTaskDelay` mandatory? If you omit `vTaskDelay`, this task will hog 100% of the CPU core execution time. Because it never voluntarily gives up control, lower-priority tasks (and even the internal watchdog timers) will starve, causing your ESP32 to crash.

Registering the Task with `xTaskCreate`

Once the function is written, you tell the operating system to run it by calling `xTaskCreate()` inside `app_main()`.

The function takes **6 parameters**.

```
xTaskCreate(  
    my_custom_task,    // 1. Pointer to the task function you wrote above  
    "My Task Name",    // 2. A text name (string) used purely for debugging  
    2048,              // 3. Stack Size (Memory allocated to this task in bytes)  
    NULL,              // 4. Parameter pointer to pass data into the task (NULL if none)  
    5,                 // 5. Priority level (Higher integer = higher urgency)  
    NULL               // 6. Task Handle (Used if you want to delete/pause the task later)  
);
```

Code:

```
#include <stdio.h>  
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
void print_message_task(void *pvParameters){  
    while(1){
```

```
        printf("Learning RTOS from scratch \n");
        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}

void counter_task(void *pvParameters){
    int count = 0;
    while(1){
        printf("Counter: %d\n", count);
        count++;
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void app_main(void){
    xTaskCreate(
        print_message_task,
        "Print Task",
        2048,
        NULL,
        5,
        NULL
    );
    xTaskCreate(
        counter_task,
        "Counter Task",
        2048,
        NULL,
        5,
        NULL
    );
}
```

Output:

```
I (253) main_task: Started on CPU0
I (253) main_task: Calling app_main()
Learning RTOS from scratch
Counter: 0
I (253) main_task: Returned from app_main()
Counter: 1
Counter: 2
Counter: 3
Learning RTOS from scratch
Counter: 4
Counter: 5
Counter: 6
Counter: 7
Learning RTOS from scratch
Counter: 8
Counter: 9
Counter: 10
Counter: 11
Learning RTOS from scratch
Counter: 12
Counter: 13
Counter: 14
Counter: 15
Learning RTOS from scratch
Counter: 16
Counter: 17
```